Tipos PRIMITIVOS y Tipo de REFERENCIA

En JS podemos distinguir dos tipo de datos:

- **primitivos** : Cuando se almacena un **valor**
- **de referencia** : Cuando el valor almacenado representa la **dirección de memoria** donde se encuentra el elemento (objeto)

Tipos de Datos PRIMITIVOS

- **Números** : Un valor de 64 bits que se emplea para cualquier tipo de dato. Coincide con el tipo **double** de JAVA El rango de posibles valores va desde `5e-324` (Puede obtenerse mediante `Number.MIN_VALUE`) a `1.7976931348623157e+308` (`Number.MAX_VALUE`).
  - Se considera un número cualquier conjunto de dígitos (que pueden incluir el separador decimal inglés) que **no vaya entrecomillado**. También se pueden especificar números en **octal** (precedidos de un `0`, por ejemplo `021` sería `17`) y en **hexadecimal** (precedidos de `0x`, por ejemplo `0x21` sería `33`). También se puede crear un número con `Number(expr)`. JS intentaá convertir la expresión a un número (`Number("12")` `12`, `Number("Hola")` `NaN`). Veremos las conversiones más adelante
  - Se puede especificar el valor **infinito** mediante el término `Infinity` (puede llevar signo) (por ejemplo `12 / 0` `Infinity`)
  - El símbolo `NaN` (**Not a Number**) **se considera un número** e indica que **un valor que debiera ser numérico no lo es** (`"Hola" * 2` `NaN`). Cualquier operación con `NaN` devuelve `NaN` y puede comprobarse si un valor es `NaN` con la función `isNaN(`*`expr`*`)`
  - Es importante destacar que **ninguna operación aritmética provocará un error, siempre se retorna un número**. En el peor de los casos, se retornará un valor `NaN`**.**
  - **NOTA:** En ECMAScript está propuesto un **nuevo tipo primitivo de datos** llamado `BigInt` (para representar valores mayores de 2^{53} `- 1`). Se puede crear añadiendo una `n` al final del número (`23n`) o con el constructor `BigInt(valor)`
- **Cadenas** : Las cadenas pueden estar formadas por conjuntos de **0** o más caracteres de 16 bits encerradas entre **comillas simples o dobles** (lo que se denomina **String Literal Notation**). Cada carácter ocupa una posición correlativa y puede ser accedido a través de un índice que comienza en **0** y mediante la propiedad `lenght` de la cadena podemos obtener su número de caracteres.

  Además el lenguaje admite las siguientes secuencias de escape :

`\t`	Tabulador
`\n`	Salto de línea
`\v`	Tabulación vertical
`\r`	Retorno de carro
`\b`	Retroceso
`\f`	Salto de página
`\'`	'
`\"`	"
`\\`	\

Actualmente en JavaScript se pueden definir cadenas ente **backticks** (``). Estas cadenas pueden ocupar varias líneas y se pueden intercalar expresiones dentro de ellas encerrándolas entre `${}` **(Template Literal Notation)(interpolación)**:

```
let nom = 'Iñigo';

alert(`Hola ${nom} ${1+3}`); // Hola Iñigo 4
```

- **Booleanos** : Un valor lógico que puede representar verdadero (**true**) o falso (**false).**
- **undefined** : Toda variable a la que no se le ha asignado un valor, tendrá como valor `undefined` (en la práctica es posible asignar el valor `undefined` a una variable pero, si se quiere indica que el valor es desconocido o no existe, es preferible emplear `null`.
- `null` : El valor `null` representa un elemento que **no existe, está vacío** o que **se desconoce su valor**. **NOTA:** Un valor `null` debe asignarse **explícitamente** para indicar que algo **no existe**. Si no se asigna ningún valor, entonces se considera `undefined`.
- **Símbolos** : Identificadores únicos para objetos (creados mediante el tipo `Symbol`)

Tipo de Datos de REFERENCIA

- **Objetos** : Tipos de datos complejos que pueden estar formados por campos y funciones

Operador typeof

El operador `typeof` retorna una cadena con el tipo de una expresión dada:

Ver código en JSFiddle

NOTA: En Javascript existe el denominado **objeto global** en el que están definidas **todas las propiedades, funciones y objetos** del sistema. En el caso de JavaScript ejecutado en el **navegador** dicho objeto es el objeto `window` mientras que en JavaScript del lado servidor, en el caso de `Node.js`, es el objeto `global`. Dado que es el objeto global **no es necesario indicarlo** para acceder a sus propiedades y métodos.

En dicho objeto existen una serie de **objetos pedefinidos** (`Math, document, screen, null...`) que el operador `typeof` retorna como `object`, y una serie de **funciones para crear objetos** mediante el operador `new` (`String, Date, Array`**...**) así como otros métodos de **utilidad** (`alert, prompt, parseFloat...`) para los que retornará **function**.

Esto mismo pasa con las propiedades y métodos del resto de los objetos (`document.writeln function`, **`document.anchors`** `object` (`document.anchors ya no se utiliza`)).

Variables

Como ya sabemos, una variable es una zona de memoria con **nombre** a la que se puede asignar (y de la que se puede recuperar) un determinado valor.

Los **identificadores** de las variables deben comenzar por una **letra** o los símbolos `$` o _ y luego pueden contener letras y/o dígitos. Como ya se ha dicho JavaScript es **sensible a mayúsculas/minúsculas** por lo que hay que tener cuidado a la hora de usar las variables.

A diferencia de otros lenguajes (como JAVA), JavaScript es un lenguaje **débilmente tipado** y esto significa que la variable **asume su tipo** en función de los datos que se le pasan. Esto quiere decir que :

- **no hay que definir el tipo de una variable a la hora de declararla**

- **una variable puede cambiar su tipo dinámicamente** en función de los valores que se le asignen

Esto, por un lado **simplifica** la creación de código pero por otro **dificulta** su depuración (ya que no nos avisará si cambiamos por error el tipo de una variable).

Para **definir** una variable, empleamos la siguiente expresión :

```
let nombreVariable [= expresión] [, nomberVariable2 [= expresión2]...]
```

NOTA: En versiones anteriores de Javascript (y en esta por motivos de compatibilidad) las variables se declaraban mediante la palabra reservada `var`. Por motivos que veremos más adelante es recomendable emplear `let` en lugar de `var` y definir todas las variables con `let` **(Introducido en ES6)**. El let tiene mejor definido el alcance de la variable, el var según el código que se programe podría dar problemas.

NOTA: La **declaración de variables** es una **instrucción en JavaScript**, mientras que la **asignación** es una **expresión JavaScript**. Aunque una expresión se puede emplear como una declaración hay diferencias (el operador **,** permite ejecutar **varias expresiones** retornando la última, pero **no varias instrucciones**):

```
> a=12, b=10;

10

> let c=1, d=2;

undefined
```

NOTA: Aunque lo veamos más adelante, la diferencia entre `let` y `var` es el **hoisting(contexto de ejecución),** (que significa que las **variables declaradas con** `var` pueden ser usadas **antes de ser declaradas**). Además las variables `var` son **globales a las funciones** en las que se declaran.

Constantes

Para definir una constante se emplea la palabra reservada **const**. A una constante sólo se le puede asignar un valor **una vez**.

```
const CTE [= expresión];
```

Conversión de Tipos de Datos

Aunque como ya se ha mencionado, JavaScript es un lenguaje débilmente tipado y las variables asumen el tipo de las expresiones, es importante entender cómo funciona la conversión de tipos de datos para evitar posibles errores.

En principio, JavaScript cuenta con **conversión implícita de tipos** de modo que intentará convertir los datos en función de los operadores empleados. Es por ello, que expresiones como `"42" * 3` retornará `126` como un número.

Pero, ¿qué ocurre con la expresión `"12" + 12`? El resultado será `"1212"` y esto es debido a que en JavaScript `+` es tanto el operador **suma** para valores numéricos como el operador **concatenación** para cadenas. En el caso indicado, el operador concatenación **tiene prioridad** sobre el operador suma.

NOTA: Dado que los operandos se evalúan de izquierda a derecha la expresión `12 + 12 + "12"` retornaría `"2412"`

En cuanto a las conversiones **implícitas** deberemos tener en cuenta lo siguiente:

- **Booleanos** : los **números distintos de** `0` se consideran `true` y el `0` se considera `false`. Las **cadenas no vacías** se consideran `true` y las vacías (`""`) `false`. Los tipos `NaN` y `null` se consideran `false`.
- **Cadenas** : los **números** son convertidos a la cadena correspondiente y en el resto de los casos (**booleanos**, `undefined, true` y `null)` se emplean sus valores como cadenas de caracteres
- **Números** : las **cadenas** son convertidas a su representación numérica tras eliminar los espacios que haya al principio o al final de las mismas, pero hay que tener en cuenta que:
  - si la cadena empieza por `0x` se asume que es un valor **hexadecimal**, si no se considera **decimal**
  - si la cadena está **vacía** se considera un valor `0`
  - si la cadena **no representa un número** se emplea como valor `NaN`

  El valor `undefined` se convierte a `NaN`

  El valor `null` se convierte a `0`

  Por otro lado, el valor booleano `true` se convierte a un `1` y el valor `false` a un `0`.

Por otro lado **JavaScript** permite el uso de **objetos** y **funciones** para realizar una **conversión explícita** de los datos.

- **Booleanos** : se puede emplear la expresión `new Boolean(expr)`.
- **Cadenas** : en este caso emplearíamos `new String(expr)`
- **Números** : además de `new Number(expr)` podemos emplear `parseFloat(expr)` para convertir a un valor con decimales o `parseInt(expr[,base])` para convertir a un entero (donde `base` es la base en la que asumimos que está la expresión, desde base 2 a base 32). En este último caso, si la expresión **tuviera decimales**, éstos **se truncan**.

Ver código en JSFiddle

Operadores

Los operadores permiten realizar operaciones sobre diferentes expresiones, dando como resultado una nueva expresión. Se categorizan en función del tipo de operandos y resultados que generan:

Artiméticos

Los operadores aritméticos son los mismos que en JAVA (y se comportan de la misma forma). De **mayor a menor prioridad** :

Operador	Descripción
`++, --`	Preincremento y predecremento
`**`	Potenciación
`*, /, %`	Multiplicación, división y resto de la división entera
`+, -`	Suma y Resta
`++, --`	Postincremento y postdecremento

NOTA: El operador `+` en su forma unaria (delante de un valor) convierte dicho valor a un número. Por tanto:

```
+true 1
```

Puede venirnos bien para convertir rápidamente cadenas a números.

Ver código en JSFiddle

Cadena

Sólo existe un operador de cadena que es la **concatenación**

Operador	Descripción
+	Concatenación de dos expresiones de cadena

Ver código en JSFiddle

Relación

Permiten comparar dos operandos.

Operador	Descripción
>	Mayor
<	Menor
>=	Mayor o igual
<=	Menor o igual
==	Igual
!=	Distinto
===	Exactamente igual (el **tipo y el valor** de ambas expresiones debe ser el mismo)
!==	Extrictamente diferentes (o el tipo o el valor o ambos son distintos)

NOTA: Cuando se comparan **datos de diferentes tipos**, JavaScript **los convierte a números**

NOTA: La diferencia entre == y === (y sus opuestos) es que, **en el segundo caso, deben coincidir tanto el valor como el tipo de los datos**:

NOTA: `null` y `undefined` con el operador == se consideran **iguales entre sí** pero **distintos a cualquier otro valor**. Con el operador === se consideran también **distintos entre sí**. Para el **resto de operadores** se convierten a números (`null` a `0` y `undefined` a `NaN`)

NOTA: Cualquier comparación con `NaN` **siempre es** `false` por lo que, cualquier valor comparado con `undefined` (exceptuando el `null` en comparación no estricta) **retornará** `false`.

Ver código en JSFiddle

Lógicos

Permiten relacionar expresiones booleanas. En orden de prioridad

Operador	Descripción
!	No. Niega el operando. Retorna `true` o `false`
&&	Y. Falso si ambos son falsos. Si la expresión es **cierta** retorna el **último elemento**, si es **falsa** el **primer** elemento **falso**
||	O. Cierto si uno es cierto. Si la expresión es **cierta** retorna el

	primer valor cierto, si es **falsa** el **último** elemento **falso**

En Javascript, los operadores booleanos pueden trabajar **con datos de cualquier tipo** que serán convertidos a **booleanos**. Se emplea **cortocircuitado de expresiones** y, como ya se ha indicado, **retorna el valor en el que se determina el resultado de la evaluación**:

NOTA: Una doble negación `!!` convierte el valor que le sigue a booleano (similar al operador `+` unario con números)

Ver código en JSFiddle

Condicional

Permiten retornar un valor en función de una expresión booleana

Operador	Descripción
`?:`	Si la expresión a la izquierda de **?** es **true**, retorna lo que hay entre **?** y **:**. Si no, lo que hay tras **:**
var edad = 5; **var texto = (edad > 0) ? "Has nacido" : "No has nacido";** **//texto valdrá "Has nacido"**	

Asignación

Permiten asignar un valor a una variable

Operador	Descripción
=	Asigna la expresión de la derecha a la variable de la izquierda del =
+=	Suma a la variable la expresión y se le asigna el resultado
-=	Resta y asignación
*=	Multiplicación y asignación
/=	División y asignación
%=	Resto y asignación

NOTA: El operador de asignación puede ser aplicado múltiples veces (de izquierda a derecha) y **devuelve** el valor asignado

```
let v1 = v2 = 3 * 2; v1 = 6, v2 = 6
```

```
let v1 = 3 * (v2 = 2); v1 = 6, v2 = 2 (los paréntesis son necesarios dado que la asignación tiene menor prioridad que el resto de los operadores vistos)
```

```
let v1 = 1;

let v2 = (++v1, v1 + 2);  v1 valdría 2 y v2 valdría 4 (el resultado de la última expresión)
```

Try / catch / finally

Al igual que en otros lenguajes de programación podemos emplear la estructura `try / catch / finally` para controlar los posibles errores en ejecución.

Ver código en JSFiddle

Variables. Ámbito

Las variables **definidas FUERA** de una **función** o **SIN INCLUIR** `var`, se consideran **GLOBALES** (variables a nivel de objeto global - `window` -). Las variables definidas **DENTRO** de una **función**, se **consideran LOCALES**. Las variables con `let` se consideran locales y, si están fuera de una función no se añaden como propiedades al objeto `window`.